

## Lecture 3: 2026-08-31 Image Filtering

*Lecturer: Tejas Gokhale**Scribe: Thanh Nam Tran*

### 3.1 Recap of Last Lecture

In the last lecture, we discussed:

- Pin-hole camera model
  - Relationship between focal length and magnification
  - Impact of aperture size on image quality and its trade-off
- Thin lens camera model
  - Gauss's lens formula, which relates  $s_o$ ,  $s_i$ , and  $f$
  - Magnification formula
  - Depth of Field (DoF) and Field of View (FoV)
- Color and wavelengths
  - Biology of color sensation
  - Tristimulus color theory
  - RGB cameras and the Bayer pattern

In today's lecture, we will discuss image filtering.

### 3.2 An Image Is a Matrix of Pixels

An image can be represented as an array of numbers called pixels. The variables  $x$  and  $y$  denote integer column and row indices. A grayscale image can be represented using one channel  $1 \times M \times N$ , while an RGB color image uses three channels  $3 \times M \times N$ .

Each pixel  $F[x, y]$  represents intensity or color information at location  $[x, y]$ . Common numerical conventions are  $[0, 1]$  for floating-point images and  $[0, 255]$  for 8-bit images.

**Note:** An RGB pixel with three 8-bit channels therefore uses 24 bits before compression or other encoding.

**Ponder This 3.1** *Is every matrix an image?*

### 3.3 Image Processing: Point Processing vs. Image Filtering

**Definition 3.1 (Image Processing)** *Image processing is a system that performs a computation on an input image  $I_{\text{in}}$  and produces an output image  $I_{\text{out}}$ .*

Examples include subtraction, inversion, brightening, darkening, smoothing, sharpening, and edge detection.

#### 3.3.1 Point Processing

**Definition 3.2 (Point Operation)** *A point operation applies a mathematical transformation to each pixel intensity independently of all other pixels.*

Let  $X$  be the original image. Example point operations include:

- Darken:  $X - 128$ ; lighten:  $X + 128$
- Lower contrast:  $X/2$ ; raise contrast:  $2X$
- Invert:  $255 - X$
- Nonlinear transformation:  $(X/255)^2 \times 255$

When an 8-bit output is required, results outside  $[0, 255]$  must be clipped or otherwise mapped back into the valid range.

Example code: <https://colab.research.google.com/drive/1Fk-L2Pad1Rus0kXcovqouZxI-lhHA6kJ?usp=sharing>

Before moving to neighborhood operations, consider the subsampled images below.

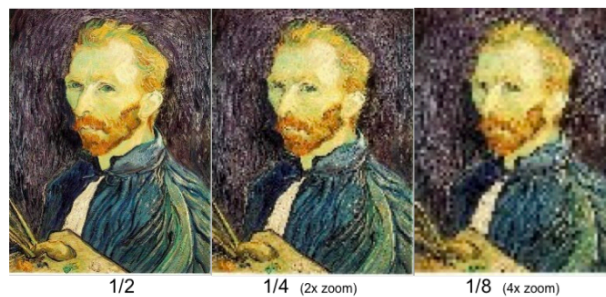


Figure 3.1: Subsampling an image

**Question:** How can we achieve this result?

**Answer:** It's simple. We discard every other row and column. However, it causes the output image to look not good.

### 3.3.2 Neighborhood Operations

**Definition 3.3 (Neighborhood Operation)** A neighborhood operation uses a kernel, mask, or window to compute each new pixel value from nearby pixels.

One example is the average filter.

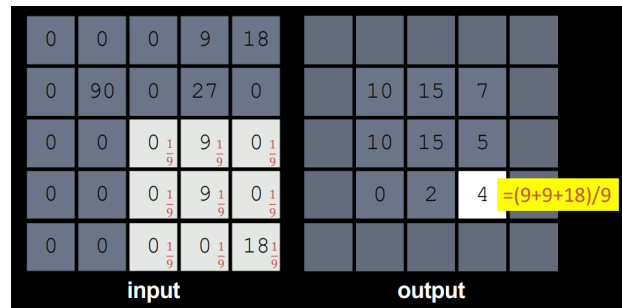


Figure 3.2: Average filtering

An average filter replaces each pixel with the arithmetic mean of its neighborhood. It is also called a box filter. A normalized  $3 \times 3$  box kernel is

$$K_{\text{box}} = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}.$$

The factor  $1/9$  makes the entries sum to one, which preserves constant image regions. The filter smooths noise but also removes sharp features and fine details.



Figure 3.3: Image before box filtering

**Note 3.1** Kernel cells do not need to have equal values. Different weights create different filtering effects.

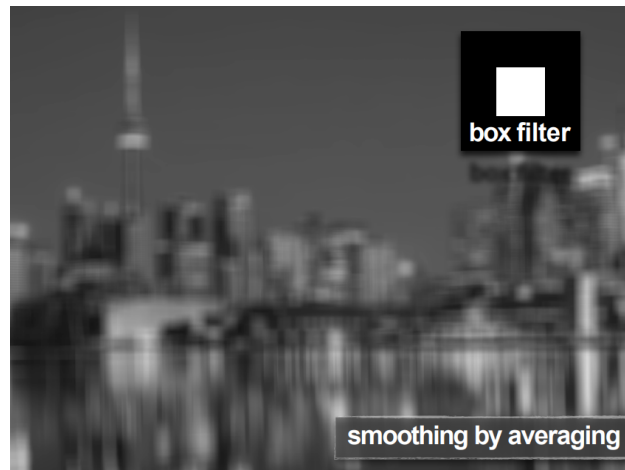


Figure 3.4: Image after box filtering

### 3.3.3 Correlation and Convolution

Correlation and convolution both slide a kernel across an image, multiply the overlapping values, and sum the products. Convolution flips the kernel horizontally and vertically, while correlation does not.

The discrete two-dimensional convolution of image  $I$  and kernel  $K$  is

$$(I * K)[x, y] = \sum_{i=-\infty}^{\infty} \sum_{j=-\infty}^{\infty} K[i, j] I[x - i, y - j]. \quad (3.1)$$

The corresponding correlation is

$$(I \star K)[x, y] = \sum_{i=-\infty}^{\infty} \sum_{j=-\infty}^{\infty} K[i, j] I[x + i, y + j]. \quad (3.2)$$

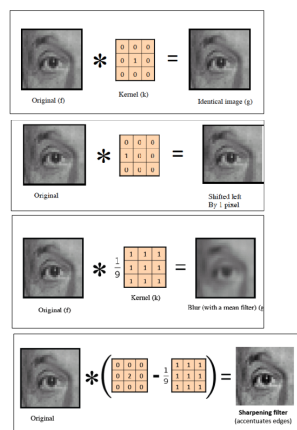


Figure 3.5: Examples of image convolution

### 3.3.4 The Gaussian Filter

**Definition 3.4 (Gaussian Filter)** A Gaussian filter is a linear smoothing filter that gives larger weights to pixels near the center of the kernel and smaller weights to pixels farther away.

The two-dimensional Gaussian function is

$$G(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right), \quad (3.3)$$

where  $\sigma$  controls the spread of the Gaussian. A larger  $\sigma$  produces stronger smoothing. A common approximation is

$$K_{\text{Gaussian}} = \frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}.$$

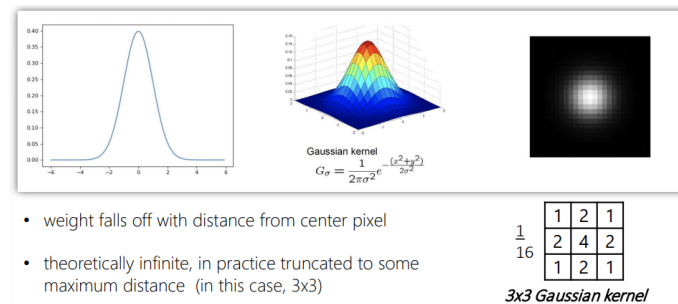


Figure 3.6: Gaussian distribution and a discrete Gaussian kernel

Gaussian filtering reduces noise and high-frequency variation while usually creating fewer harsh artifacts than a box filter.

### 3.3.5 Edge Detection in Images

An image edge is a location where intensity changes rapidly. Edges are useful because much of an object's shape information is concentrated around its boundaries, making edge representations more compact than raw pixel values.

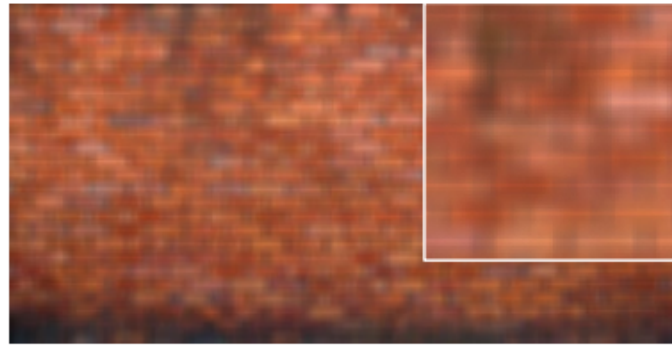
Rapid intensity changes can be detected using first derivatives. The image gradient is

$$\nabla I = \begin{bmatrix} \frac{\partial I}{\partial x} \\ \frac{\partial I}{\partial y} \end{bmatrix} = \begin{bmatrix} I_x \\ I_y \end{bmatrix}. \quad (3.4)$$

An example derivative kernel is



7x7 Gaussian



7x7 box

Figure 3.7: Comparison of Gaussian and box filtering

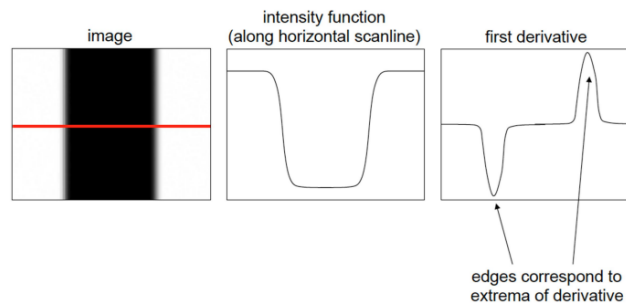


Figure 3.8: An edge as a rapid change in image intensity

$$K_x = \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}.$$

This kernel measures horizontal intensity change and therefore responds strongly to vertical edges.

After computing horizontal and vertical derivatives, edge strength and direction can be calculated as

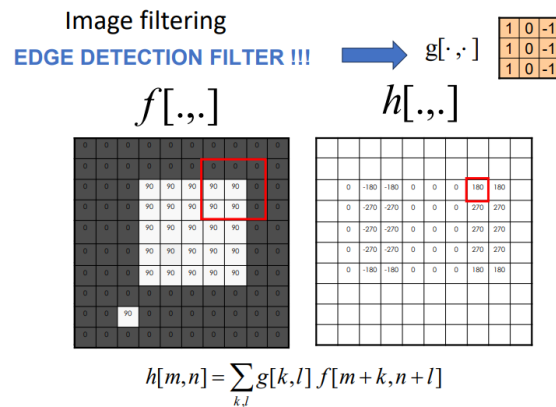


Figure 3.9: Edge detection using a derivative filter

$$\|\nabla I\| = \sqrt{I_x^2 + I_y^2}, \quad \theta = \text{atan2}(I_y, I_x). \quad (3.5)$$

The magnitude  $\|\nabla I\|$  measures edge strength, while  $\theta$  gives the direction of greatest intensity increase.

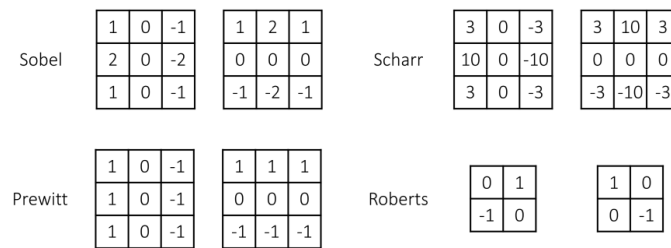


Figure 3.10: Common edge-detection filters

There are different edge detection filters

- **Sobel:** Provides a useful balance between smoothing, speed, and edge detection, but may produce thicker edges and has lower rotational accuracy than Scharr.
- **Prewitt:** Simple and computationally inexpensive, but more sensitive to noise and less rotationally accurate than Sobel or Scharr.
- **Scharr:** Provides improved rotational accuracy for a  $3 \times 3$  kernel, but its larger coefficients may require scaling or normalization.
- **Roberts:** Uses a small  $2 \times 2$  kernel and provides precise edge localization, but is highly sensitive to noise.

The general gradient-computation procedure is:

1. Select horizontal and vertical derivative filters.

2. Convolve each filter with the image to compute  $I_x$  and  $I_y$ .
3. Combine the derivatives to compute gradient magnitude and direction.

### 3.3.6 Sharpening

Sharpening increases local contrast around edges. One common sharpening kernel is

$$K_{\text{sharp}} = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}.$$

The center pixel is emphasized while neighboring values are subtracted.

### 3.3.7 Thresholding

Thresholding converts an intensity or gradient image into a binary image.

The equation is

$$g(m, n) = \begin{cases} 255, & f(m, n) > A \\ 0 & \text{otherwise} \end{cases}$$



Figure 3.11: Thresholding Example

**Ponder This 3.2** *Does thresholding make storage efficient? By how much (in this example)?*

**Answer:** A grayscale pixel can have 256 values, from 0 to 255. It needs 8 bits. After thresholding, each pixel is only black or white. It needs only 1 bit. If done correctly, it can help with storage efficiency. In the example, we can assume if the original is in 8-bit, then thresholding helps save 8 times the storage.